Pre-Logon VPN (PLAP / Start Before Logon) Troubleshooting & Diagnostics Toolkit

Implementation Solution Guide

Version 1.0.0 · 2026-07-08

Contents

Executive Summary 4

The problem . . . 4

What this toolkit delivers . . . 4

How it isolates the failure fast . . . 4

The vendor boundary (by design) . . . 5

The bottom line . . . 5

Solution Overview 5

What this toolkit is . . . 5

The two mechanisms it diagnoses . . . 5

The named objects this toolkit looks for . . . 6

The five diagnostic tools . . . 6

Where the vendor boundary sits . . . 7

How to use the rest of this toolkit . . . 7

Requirements 8

R1. Access to a device that exhibits the problem . . . 8

R2. Local administrator rights on the failing device . . . 8

R3. Microsoft Intune admin center read access . . . 9

R4. Microsoft Graph read access (for the Intune check script) . . . 9

R5. Knowledge of the intended mechanism and gateway . . . 9

R6. A known-good reference device (baseline) . . . 10

R7. VPN head-end / gateway vantage point (evidence-only) . . . 10

Diagnostic Workflow 10

Phase 1 — Collect (always start here) . . . 11

Phase 2 — Analyze (run the focused check for the flagged area) . . . 11

Phase 3 — Remediate (in the owning plane) . . . 13

Phase 4 — Verify (re-run until clean) . . . 14

Testing — Confirming the Toolkit and the Fix 14

Test matrix overview . . . 14

T1 — Collector runs and writes a report . . . 15

T2 — Known-good device reports healthy (no false positives) . . . 15

T3 — Known-bad device reports the true fault (no false negatives) . . . 15

T4 — SBL/PLAP checks are accurate . . . 15

T5 — Device-tunnel checks are accurate . . . 15

T6 — Certificate deep-check is accurate . . . 15

T7 — Intune check finds and correctly scopes objects . . . 16

T8 — Post-fix re-run comes back clean . . . 16
Overall exit criteria . . . 16
Troubleshooting — Pre-Logon VPN Symptom → Cause → Fix Matrix 16
§SBL/PLAP — the Network sign-in tile is missing . . . 17
§SBL/PLAP — the tile appears but the connection fails . . . 17
§Device tunnel — never establishes at boot . . . 18
§Certificate — the machine / VPN certificate is the fault . . . 18
§Cross-cutting — affects either mechanism . . . 19
§Script / tooling issues . . . 19
When to stop diagnosing and escalate to the gateway vendor . . . 20
Best Practices — Repeatable Pre-Logon VPN Triage 20
1. Name the intended mechanism before you touch anything . . . 20
2. Collect first, reason second . . . 20
3. Check preconditions in a fixed order . . . 20
4. Keep a known-good baseline current . . . 21
5. Get the assignment scope right — device vs user . . . 21
6. Read-only discipline . . . 21
7. Respect the vendor boundary . . . 21
8. Least-privilege, auditable access . . . 22
9. Close the loop . . . 22
Deliverables Summary 22
Documentation . . . 22
PowerShell scripts (all read-only) . . . 23
What is intentionally deferred to the gateway vendor . . . 24
How the pieces fit together . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary

The problem

Pre-logon VPN — a tunnel that comes up **before** a user signs in, so a device can reach a domain controller and a first-time user can log on off the corporate network — is powerful and fragile. When it stops working, the failure is genuinely hard to place: the **Network sign-in** option never appears, the device tunnel silently never connects, or authentication is rejected with no obvious reason. Worse, there are **two different mechanisms** in play — Cisco **Start Before Logon (SBL/PLAP)** and the Windows Always On VPN **device tunnel** — each with its own strict preconditions, and a break in any one precondition looks like a total failure. Teams burn hours per incident guessing, and often "fix" the wrong mechanism entirely.

What this toolkit delivers

A **structured, precondition-by-precondition diagnostic workflow** plus **five read-only PowerShell tools** that together isolate the missing precondition in minutes instead of hours — for **both** mechanisms. It is a **troubleshooting/diagnostics** product: a self-guided playbook you keep and reuse on every incident, not a fresh deployment guide.

The toolkit includes:

- **Solution Overview** — the two pre-logon mechanisms and every precondition each one needs (the SBL

module + `vpnplap64.dll` PLAP provider + `UseStartBeforeLogon`; the device tunnel's SKU/join, single-tunnel, IKEv2, machine cert, and AlwaysOn rules).

- **Requirements Guide** — the device access, admin rights, and Intune/Graph read access to have ready,

each with how to verify it.

- **Diagnostic Workflow** — the ordered **Collect** → **Analyze** → **Remediate** → **Verify** process and a

symptom → tool map that sends you straight to the right check.

- **Five read-only tools** — a master collector plus focused checks for SBL/PLAP, the device tunnel,

the certificate, and the Intune object/scope audit. All **read-only**.

- **Testing & Troubleshooting guides** — gates that prove the tool reports a known-good device healthy

and a known-bad device's true fault, and a comprehensive symptom → cause → fix matrix covering both mechanisms.

How it isolates the failure fast

Each mechanism's preconditions are checked in a **fixed order**, so the **first FAIL is the whole reason**: no SBL tile almost always means a missing module or an unset `UseStartBeforeLogon`; a dead device tunnel almost always means the wrong SKU/join, more than one tunnel, a non-IKEv2 type, or a profile mis-scoped to a **user** group instead of a **device** group. The toolkit names the precondition and hands you the remedy.

The vendor boundary (by design)

The **Windows, Intune, Entra, and certificate side is authored at exact detail** — that's where the scripts and precision live. The **VPN head-end / gateway internals are deferred to the vendor's official documentation**: the toolkit tells you exactly *what evidence to read* at the gateway (did the attempt arrive, the rejection reason, the issuing-CA it must trust) and hands you to the vendor doc for the exact console clicks, so you never chase a fabricated menu path.

The bottom line

The missing precondition behind a broken pre-logon VPN — SBL/PLAP or device tunnel — isolated in minutes. A structured workflow and read-only scripts you keep and reuse on every incident, at a fraction of the cost of consulting on each outage.

Solution Overview

What this toolkit is

This is a **diagnostics toolkit**, not a deployment guide. It assumes you already run — or are supposed to run — **pre-logon VPN** on Windows: a tunnel that comes up **before** (or without) an interactive user sign-in, so the device can reach a domain controller, apply Group Policy, and let a first-time user log on over the corporate network. Something in that flow has stopped working — the network sign-in option never appears, the tunnel never comes up, or authentication is rejected — and the fault is hiding somewhere across several cooperating preconditions. This toolkit collects the device and Intune state, checks each precondition, and points you at the fix.

The core idea: **a pre-logon VPN failure is almost never "the VPN is broken."** It is one specific missing precondition — a module not installed, a flag not set, a certificate with the wrong usage, a profile assigned to the wrong context — cascading into a symptom that looks like a total failure. If you can name the missing precondition, you can name the fix. The whole toolkit exists to help you name it fast.

The two mechanisms it diagnoses

Windows offers **two different ways** to bring a VPN up before logon, and they fail for different reasons. This toolkit diagnoses **both** and tells you which one you are (or should be) using.

Mechanism 1 — Cisco Start Before Logon (SBL / PLAP)

Cisco Secure Client's **Start Before Logon (SBL)** shows a **Network sign-in** option on the Windows logon screen (the PLAP — *Pre-Logon Access Provider* — credential provider), so a user connects the VPN *before* signing in. The "Network sign-in" tile appears **only if both** of these are true:

- The Cisco Secure Client **VPN profile** carries `<UseStartBeforeLogon>true</UseStartBeforeLogon>`, **and**
- The **Start Before Logon module** MSI is installed (Core VPN must be installed **first**).

The component that renders the tile is the 64-bit PLAP DLL `vpnplap64.dll`, registered as a Windows **credential provider** under

`HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Authentication\Credential Providers`. If either precondition is missing — module absent, flag not set, or the provider not registered — the tile never shows, and users on a fresh machine can't log on off-network.

Mechanism 2 — Windows Always On VPN (AOVPN) device tunnel

The **device tunnel** is a machine-context VPN that connects automatically at boot using a **machine certificate** — no user needed. It is the native Windows way to get pre-logon connectivity. It has strict preconditions:

- **Only one** device tunnel is allowed **per device**.
- The device must be **domain-joined or hybrid-joined** and running **Windows Enterprise or Education**.
- The connection must be **IKEv2** (device tunnels do not support other tunnel types).
- A **machine certificate** must be in `LocalMachine\My` with the **Client Authentication** EKU, and

the issuing CA must be trusted by the VPN gateway.

- The profile (delivered via the **VPNv2 CSP** as **ProfileXML**) must carry `DeviceTunnel=true`

and `AlwaysOn=true`, and the tunnel must be visible via `Get-VpnConnection -AllUserConnection`.

If any precondition is off — wrong SKU/join type, a user-context profile instead of device-context, the wrong certificate EKU, or more than one device tunnel — the tunnel silently never establishes.

The named objects this toolkit looks for

Diagnosis is comparison, so the toolkit reuses the **exact object names** a matching deployment uses. Reuse these verbatim (they appear in the scripts and the troubleshooting matrix):

Object	Name
Device-tunnel VPN profile (VPNv2 CSP / ProfileXML)	`Pre-Logon VPN – Device Tunnel – Profile`
SBL module app (Start Before Logon)	`Pre-Logon VPN – SBL Module – Win32`
SBL-enabling VPN profile (UseStartBeforeLogon)	`Pre-Logon VPN – SBL – Profile`
Trusted-root certificate profile	`Pre-Logon VPN – Trusted Root`
Machine-certificate profile (SCEP/PKCS)	`Pre-Logon VPN – Device Cert (SCEP)`
Pilot / test **device** group	`PILOT – Pre-Logon VPN Devices`

The five diagnostic tools

All five are **read-only**. They observe; they never remediate.

- `PLAPDX-Collect-Diagnostics.ps1` — the entry point. A single on-device master collector that

snapshots every precondition for **both** mechanisms (OS SKU/build, join type, Secure Client version, `vpnplap64.dll` + PLAP provider registration, the `UseStartBeforeLogon` value, the AOVPN device tunnel, the machine certificate) and writes one report.

• `PLAPDX-Check-PlapSbl.ps1` — focused **SBL/PLAP** checks with PASS/FAIL per precondition and the

exact remedy for each FAIL.

• `PLAPDX-Check-DeviceTunnel.ps1` — focused **device-tunnel** checks (SKU/join eligibility, exactly

one device tunnel, IKEv2, machine cert, AlwaysOn) with PASS/FAIL + remedy.

• `PLAPDX-Check-Certificate.ps1` — a certificate deep-check (present, not expired, Client

Authentication EKU, chains to a trusted root, private key present), with optional expected-issuing-CA confirmation.

• `PLAPDX-Check-Intune.ps1` — a read-only Microsoft Graph check that confirms the expected Intune

objects exist, are assigned, and are **scoped to the right context** (a device-tunnel profile assigned to a **device** group, not a user group).

Where the vendor boundary sits

This toolkit authors the **Windows, Intune, Entra, and certificate side** at exact, click-by-click detail — that is where your read-only scripts and portal checks live. The **VPN head-end / gateway internals are deferred to the vendor's official documentation**: the Cisco Secure Client profile editor and head-end (ASA / Firepower FTD) config, and the RRAS / third-party IKEv2 gateway configuration. The toolkit tells you *what* the device and Intune must present (the certificate, the profile flags, the module); *how* the gateway is configured to accept it is the vendor's domain and your gateway admin's.

How to use the rest of this toolkit

• `02-requirements.md` — the read access and vantage points you need before you can diagnose.

Confirm these first; assembling access mid-incident wastes the outage.

• `03-implementation.md` **(Diagnostic Workflow)** — the ordered **Collect → Analyze → Remediate →

Verify **workflow and the** symptom → tool** map. Start here during an incident.

• `05-testing.md` — pass/fail gates to confirm the toolkit runs correctly and that a fix actually

cleared the failing precondition.

• `06-troubleshooting.md` — the comprehensive **symptom** → **likely cause** → **fix** matrix for

pre-logon VPN across **both** mechanisms. This is the heart of the product.

• `07-best-practices.md` — repeatable triage habits, evidence collection, and clean escalation.

Standing constraint for the whole toolkit: every script and check here is **read-only**. Diagnostics observe; they never change the device, the tenant, or the gateway. You decide the fix; the toolkit only tells you where it is. The workflow adapts the category-standard **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model to a diagnostic shape: **Collect** → **Analyze** → **Remediate** → **Verify.**

Requirements

Before you can diagnose a pre-logon VPN failure — Cisco Start Before Logon (SBL/PLAP) or a Windows Always On VPN device tunnel — you need read access to each place that holds evidence, and access to a device that exhibits the problem. This section lists what to have ready. Each item follows **Description / Why / Verify / Implement if missing**.

Gather these *before* an incident where possible — assembling access mid-incident wastes the outage. Everything here is **read-only**; you never need write/remediation rights to run this toolkit.

> *Naming convention used throughout this toolkit (reuse these names verbatim): - Device-tunnel VPN profile (VPNv2 CSP / ProfileXML):* `Pre-Logon VPN – Device Tunnel – Profile` *- SBL module app:* `Pre-Logon VPN – SBL Module – Win32`*; SBL-enabling VPN profile:* `Pre-Logon VPN – SBL – Profile` *- Trusted-root certificate profile:* `Pre-Logon VPN – Trusted Root`*; machine-certificate profile:* `Pre-Logon VPN – Device Cert (SCEP)` *- Pilot / test **device** group in Entra:* `PILOT – Pre-Logon VPN Devices`
>
> ***Government / sovereign-cloud note:** if the tenant is **GCC High / DoD**, the admin portals differ (*`https://endpoint.microsoft.us`*, *`https://entra.microsoft.us`*, sign-in via* `https://login.microsoftonline.us`*) and feature/endpoint parity can lag Commercial. The device-side checks are identical, but **verify the portal steps and object availability for GCC High** in your sovereign cloud before acting on the Intune-side checks.*

R1. Access to a device that exhibits the problem

Description: Local (or remote-management) access to the enrolled **Windows** device where pre-logon VPN fails — enough to run the device-side scripts and read its certificate store, registry, services, and VPN configuration.

Why: Most pre-logon preconditions have a **device-side truth** the cloud cannot see: whether the `vpnplap64.dll` PLAP provider is actually registered, whether the machine certificate is present with a private key and the Client Authentication EKU, and whether a device tunnel actually exists. The scripts `PLAPDX-Collect-Diagnostics.ps1`, `PLAPDX-Check-PlapSbl.ps1`, `PLAPDX-Check-DeviceTunnel.ps1`, and `PLAPDX-Check-Certificate.ps1` run **on this device**.

Verify: You can open a PowerShell session on the device and run `Get-ChildItem Cert:\LocalMachine\My` and `Get-VpnConnection -AllUserConnection` and see output.

Implement if missing: Arrange local admin access (needed to read `LocalMachine\My` and the Credential Providers registry key), a remote session (RDP / remote PowerShell), or have the end user run the read-only collector and return the report. No changes are made to the device.

R2. Local administrator rights on the failing device

Description: Local administrator (or SYSTEM) rights on the device, to read the machine certificate store `Cert:\LocalMachine\My`, the all-user VPN connections, and the Credential Providers registry key.

Why: The device-tunnel machine certificate lives in `LocalMachine\My`, the device tunnel is an all-user connection, and the PLAP credential-provider registration is under `HKLM`. A standard user can't read these reliably, so a check would report false negatives.

Verify: In an elevated PowerShell session, `Get-ChildItem Cert:\LocalMachine\My` returns the machine certs and `Get-Item 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Authentication\Credential Providers'` succeeds.

Implement if missing: Run the device-side scripts **elevated** (Run as administrator) or as SYSTEM via your management tooling. The scripts read only; elevation is for visibility, not change.

R3. Microsoft Intune admin center read access

Description: Sign-in to the Microsoft Intune admin center (`https://intune.microsoft.com`; GCC High: `https://endpoint.microsoft.us`) with at least a read-only role that can view apps, device configuration profiles, certificate profiles, and their **assignments**.

Why: Whether the SBL module app, the device-tunnel profile, the SBL-enabling VPN profile, the trusted-root profile, and the SCEP/PKCS certificate profile are **present, assigned, and scoped to the right context** is only visible here. A device-tunnel profile assigned to a *user* group instead of a *device* group is a classic silent failure.

Verify: Browse to the Intune admin center → **Apps** and **Devices > Configuration** and confirm you can read each object's **Properties** and **Assignments**.

Implement if missing: In Entra (`https://entra.microsoft.com`) → **Roles & admins**, assign the operator the **Global Reader** role (tenant-wide read, no change rights) or the built-in **Intune Administrator** role. Prefer read-only.

R4. Microsoft Graph read access (for the Intune check script)

Description: The ability to run `PLAPDX-Check-Intune.ps1` against Microsoft Graph with the read-only scopes `DeviceManagementApps.Read.All`, `DeviceManagementConfiguration.Read.All`, `Group.Read.All`, and `Directory.Read.All`.

Why: The script reads the same app/profile/assignment state you would click through in the portal — so it can confirm each expected object exists, is assigned to `PILOT – Pre-Logon VPN Devices`, and (the key check) that the **device-tunnel profile is assigned to a device group, not a user group**.

Verify: Run `Connect-MgGraph -Scopes "DeviceManagementApps.Read.All","DeviceManagementConfiguration.Read.All","Group.Read.All","Directory.Read.All"` and confirm consent succeeds; `Get-MgContext` shows the scopes granted.

Implement if missing: Install the Microsoft Graph PowerShell SDK (`Install-Module Microsoft.Graph -Scope CurrentUser`). Consent to the delegated **read** scopes above at first connect — no write consent is required or requested.

R5. Knowledge of the intended mechanism and gateway

Description: A recorded statement of **which** pre-logon mechanism this device is supposed to use (Cisco SBL/PLAP or an AOVPN device tunnel), the **gateway/head-end address**, the **expected issuing CA** for the machine/VPN certificate, and — for SBL — the expected Cisco Secure Client version.

Why: Diagnosis is comparison. Knowing the intended mechanism stops you from "fixing" a device tunnel on a machine that was only ever meant to use SBL (and vice-versa), and the expected issuing CA turns "is this certificate right?" into a one-line match.

Verify: You can state, for this device: mechanism (SBL vs device tunnel), gateway FQDN, expected issuing-CA name, and the expected profile name from the naming table above.

Implement if missing: Obtain the design decision from whoever owns the pre-logon VPN deployment (the matching **deployment** product records these). If nobody can say which mechanism is intended, that ambiguity is itself a finding — pick one design and standardize.

R6. A known-good reference device (baseline)

Description: At least one Windows device on which pre-logon VPN **currently works** for the same mechanism, plus its expected values (issuer/template of the machine cert, profile name, gateway).

Why: A working reference turns "is this profile correct?" into a diff and "is the certificate right?" into "does it match the working one's issuer and template?" It also proves the *service* (CA, gateway) is up, isolating the fault to the failing device.

Verify: Confirm one device brings up the pre-logon tunnel / shows the Network sign-in tile right now; record its cert issuer/template, profile name, and gateway.

Implement if missing: Use a device from `PILOT – Pre-Logon VPN Devices` as the known-good reference. If **nothing** currently works, the fault is likely a shared/service break (issuing CA, gateway, or a fleet-wide assignment error) rather than a single device — note that and start from the Intune check (`PLAPDX-Check-Intune.ps1`).

R7. VPN head-end / gateway vantage point (evidence-only)

Description: Read access to, or a hand-off with, whoever administers the **VPN gateway** (Cisco ASA/FTD for SBL, or the RRAS / third-party IKEv2 gateway for the device tunnel), to read whether the device's authentication attempt **reached** the gateway and why it was accepted or rejected.

Why: Whether the gateway **trusts the issuing CA**, whether it received the connection attempt, and the exact rejection reason are only visible at the head-end. Without this, a "cert looks perfect device-side but the gateway rejects it" case is guesswork.

Verify: You (or the gateway admin on the call) can locate the failing device's connection attempt in the gateway logs by time/identity.

Implement if missing: Arrange gateway read access or an agreed hand-off where the gateway admin pulls the log for a timestamp you provide. **Exact gateway console navigation and trust-store configuration are deferred to the vendor's official documentation** — this toolkit names the evidence to read (did it arrive, what was the rejection reason); the vendor doc covers where it lives.

Diagnostic Workflow

This is the core deliverable: the ordered workflow for a pre-logon VPN incident. It adapts the category-standard **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model to a diagnostic shape:

Collect → Analyze → Remediate → Verify.

During an incident, start here. Work it in order — **Collect once**, then **Analyze** the area the symptom points to with the focused `Check-*` tools, **Remediate** in the owning plane (referencing the matching deployment product), and **Verify** by re-running the tools until the failing precondition is clean.

> *Names used verbatim (from `01-solution-overview.md` / `02-requirements.md`):* `Pre-Logon VPN – Device Tunnel – Profile`, `Pre-Logon VPN – SBL Module – Win32`, `Pre-Logon VPN – SBL – Profile`, `Pre-Logon VPN – Trusted Root`, `Pre-Logon VPN – Device Cert (SCEP)`, `PILOT – Pre-Logon VPN Devices`.

Phase 1 — Collect (always start here)

Run the master collector **once** on the failing device to snapshot every precondition for **both** mechanisms into one report — so you spend the incident reasoning, not gathering.

```
powershell -ExecutionPolicy Bypass -File .\PLAPDX-Collect-Diagnostics.ps1 -OutFile .\plapdx-report.txt
```

Run it **elevated** (R2) so it can read `LocalMachine\My`, the all-user VPN connections, and the Credential Providers registry key. The report captures:

- **OS** SKU + build (device tunnel needs Enterprise/Education);
- **Join type** via `dsregcmd /status` (device tunnel needs domain-joined / hybrid-joined);
- **Cisco Secure Client** version (SBL relevance);
- `vpnplap64.dll` presence and **PLAP credential-provider registration** (SBL tile);
- the `UseStartBeforeLogon` value in the Secure Client VPN profile;
- the **AOVPN device tunnel** (`Get-VpnConnection -AllUserConnection`) and its `DeviceTunnel`/`AlwaysOn`

flags and tunnel type;

- the **machine certificate** (`LocalMachine\My`: EKU + expiry + private key).

The report ends with a **flagged mechanism and symptom**. Read it, then jump to the matching branch in **Phase 2**.

Gate (Collect): the report wrote successfully and names the intended mechanism's state. If the device can't run PowerShell elevated, resolve R1/R2 first.

Phase 2 — Analyze (run the focused check for the flagged area)

Pick the branch by **symptom**. Each focused tool prints **PASS/FAIL per precondition** and the exact remedy for each FAIL.

Symptom → tool map

Symptom	Mechanism	Run this tool	Precondition it isolates
The **Network sign-in** / PLAP tile never appears on the logon screen	SBL/PLAP	`PLAPDX-Check-PlapSbl.ps1`	Module installed · `vpnplap64.dll` present · provider registered · `UseStartBeforeLogon=true`
Tile appears, but it's a device meant to use the **device tunnel**	Device tunnel	`PLAPDX-Check-DeviceTunnel.ps1`	Wrong mechanism in use — see remediation
The **device tunnel never comes up** at boot	Device tunnel	`PLAPDX-Check-DeviceTunnel.ps1`	SKU/join eligible · exactly one device tunnel · IKEv2 · machine cert · AlwaysOn
Tunnel/tile is there, but **authentication is rejected**	Either	`PLAPDX-Check-Certificate.ps1`	Cert present · not expired · Client Auth EKU · chains to trusted root · private key
A **first-time user can't log on off-network** (no DC reachable)	Either	`PLAPDX-Check-PlapSbl.ps1` **or** `PLAPDX-Check-DeviceTunnel.ps1`	Whichever mechanism should provide the pre-logon path
Nothing works fleet-wide, or you suspect mis-assignment	Either	`PLAPDX-Check-Intune.ps1`	Objects exist · assigned · **device-tunnel profile on a device group**

Branch A — SBL/PLAP tile missing

```
powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-PlapSbl.ps1
```

It checks, in order: is the **Start Before Logon module** installed (Core VPN first), is `vpnplap64.dll` present, is the **PLAP credential provider registered** under `HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Authentication\Credential Providers`, is the Secure Client **VPN profile present**, and does it carry `<UseStartBeforeLogon>true</UseStartBeforeLogon>`. The first FAIL is almost always the whole reason the tile is missing.

Branch B — device tunnel won't establish

```
powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-DeviceTunnel.ps1
```

It checks: **OS SKU/join type eligible** (Enterprise/Education + domain/hybrid-joined), **exactly one** device tunnel exists, it is **IKEv2**, it carries `DeviceTunnel=true`/`AlwaysOn=true`, and a valid **machine certificate** backs it. More than one device tunnel, a non-IKEv2 type, or a user-context profile are the usual causes.

Branch C — authentication rejected (certificate deep-check)

```
powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-Certificate.ps1 -CaThumbprint <expected-issuing-CA
```

It reports, for each candidate machine certificate: **present**, **not expired**, **Client Authentication EKU present**, **chains to a trusted root**, **private key present**, and — if you pass `-CaThumbprint` — whether it was issued by the **expected CA**. A cert that passes device-side but is still rejected points at the gateway's CA trust (R7, vendor-deferred).

Branch D — Intune object / scoping audit

```
powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-Intune.ps1
```

It confirms the expected objects exist and are assigned to `PILOT – Pre-Logon VPN Devices`, and flags the classic mis-scope: a **device-tunnel profile assigned to a user group** (it must target a **device** group, because the device tunnel has no user at connect time). Read-only Graph scopes only.

Gate (Analyze): the flagged tool reports a specific **FAIL** with a named remedy (or all PASS — in which case the fault is at the gateway/service, R7). Do not guess past a FAIL; fix it and re-check.

Phase 3 — Remediate (in the owning plane)

The toolkit is **read-only** — it never changes the device, tenant, or gateway. Apply the indicated fix in the plane that owns it, using the matching **deployment** product for the exact click-by-click steps. Typical remedies by finding:

Finding	Where to fix	Typical remedy
SBL module not installed	Intune (app)	Deploy `Pre-Logon VPN – SBL Module – Win32` (Core VPN first) to the device group
`UseStartBeforeLogon` not set	Intune (VPN profile)	Set `<UseStartBeforeLogon>true</UseStartBeforeLogon>` in `Pre-Logon VPN – SBL – Profile` and re-push
PLAP provider not registered	Device	Repair/reinstall the SBL module so `vpnplap64.dll` registers
Wrong OS SKU / join type (device tunnel)	Device / provisioning	Move to Enterprise/Education + domain/hybrid join, or use SBL instead
More than one device tunnel	Device (management plane)	Remove the extra device tunnel; only one is allowed
Non-IKEv2 device tunnel	Intune (ProfileXML)	Reissue `Pre-Logon VPN – Device Tunnel – Profile` as IKEv2
Machine cert missing / wrong EKU / expired	Intune (cert)	Fix `Pre-Logon VPN – Device Cert (SCEP)` (Client Auth EKU, renewal) after the trusted root
Issuing CA not trusted by the gateway	Gateway (vendor)	Add the issuing CA to the head-end trust store — **deferred to vendor docs**
Device-tunnel profile on a user group	Intune (assignment)	Re-assign `Pre-Logon VPN – Device Tunnel – Profile` to a **device** group

Because several of these touch outward-facing/shared services (the issuing CA, the gateway, broadly assigned profiles), remediate under **change control** — see `07-best-practices.md`.

Phase 4 — Verify (re-run until clean)

After a fix, **re-run the focused tool** for the branch you worked, then re-run the collector to prove the whole picture is clean and no new gap opened:

```
powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-PlapSbl.ps1        # or -DeviceTunnel / -Certifica
powershell -ExecutionPolicy Bypass -File .\PLAPDX-Collect-Diagnostics.ps1  # confirm the full picture
```

Then prove it **on the logon screen / at boot**: reboot the device and confirm the **Network sign-in** tile appears (SBL) or the **device tunnel connects before logon** (device tunnel), and that a first-time user can sign in off-network. Confirm each gate in `05-testing.md`.

Gate (Verify): the failing precondition now PASSes, the collector shows no other FAIL, and the device demonstrably brings up pre-logon connectivity. Only then is the incident closed. Record the root-cause precondition on the ticket (see `07-best-practices.md`, closing the loop).

Testing — Confirming the Toolkit and the Fix

Two things get tested here. First, that the **toolkit itself runs correctly** — it must report a known-good device as healthy and a known-bad device as failing, so you can trust its verdicts. Second, that a **fix actually cleared** the failing precondition. Each check has an explicit **pass/fail gate**. All checks here are **read-only** — you observe; you never force a state.

> *Reuse names verbatim:* `Pre-Logon VPN – Device Tunnel – Profile`, `Pre-Logon VPN – SBL Module – Win32`, `Pre-Logon VPN – SBL – Profile`, `Pre-Logon VPN – Trusted Root`, `Pre-Logon VPN – Device Cert (SCEP)`, `PILOT – Pre-Logon VPN Devices`.

Test matrix overview

#	Test	Where	Pass gate
T1	Collector runs and writes a report	Any device	Report written; mechanism + symptom named
T2	Known-good device reports healthy	Reference device (R6)	Intended mechanism all PASS
T3	Known-bad device reports the fault	Failing device	The tool FAILs on the true missing precondition
T4	SBL/PLAP checks are accurate	SBL device	Tile-present ⇔ all SBL preconditions PASS
T5	Device-tunnel checks are accurate	Device-tunnel device	Tunnel-up ⇔ all device-tunnel preconditions PASS
T6	Certificate deep-check is accurate	Any device	Verdict matches the actual cert state

#	Test	Where	Pass gate
T7	Intune check finds + scopes objects	Operator workstation	Objects found, assigned, correctly scoped
T8	Post-fix re-run comes back clean	Fixed device	Failing precondition now PASS; tile/tunnel works

T1 — Collector runs and writes a report

Steps: On any Windows device, run `PLAPDX-Collect-Diagnostics.ps1 -OutFile .\plapdx-report.txt` elevated. **PASS:** The console shows the per-area summary and `plapdx-report.txt` is written, ending with a flagged mechanism and symptom. **FAIL:** The script errors or can't read `LocalMachine\My` / the Credential Providers key → run elevated (R2) and re-test.

T2 — Known-good device reports healthy (no false positives)

Steps: Run the focused tool for the **working** reference device's mechanism (`PLAPDX-Check-PlapSbl.ps1` or `PLAPDX-Check-DeviceTunnel.ps1`). **PASS:** Every precondition for the intended mechanism reports **PASS**, matching the fact that the device brings up pre-logon connectivity. **FAIL:** A precondition reports FAIL on a device that demonstrably works → the check's expectation is mis-set (e.g. wrong expected issuer); reconcile before trusting it on the failing device.

T3 — Known-bad device reports the true fault (no false negatives)

Steps: On the failing device, run the tool the symptom points to (see the `03-implementation.md` symptom→tool map). **PASS:** The tool FAILs on the **actual** missing precondition (confirmed against `06-troubleshooting.md`). **FAIL:** The tool reports all PASS on a device that clearly doesn't work → the fault is beyond the device (gateway CA trust / service, R7) or a check gap; escalate with evidence.

T4 — SBL/PLAP checks are accurate

Steps: On an SBL device, run `PLAPDX-Check-PlapSbl.ps1` and compare to the logon screen. **PASS:** All four SBL preconditions PASS **iff** the **Network sign-in** tile actually appears; any FAIL corresponds to a genuinely missing module / flag / registration. **FAIL:** Tile present but a precondition FAILs (or vice-versa) → investigate the discrepancy per `06-troubleshooting.md §SBL/PLAP`.

T5 — Device-tunnel checks are accurate

Steps: On a device-tunnel device, run `PLAPDX-Check-DeviceTunnel.ps1` and compare to `Get-VpnConnection -AllUserConnection` and boot behavior. **PASS:** SKU/join, single-tunnel, IKEv2, machine cert, and AlwaysOn all PASS **iff** the device tunnel connects before logon. **FAIL:** Tunnel up but a precondition FAILs (or the reverse) → investigate per `06-troubleshooting.md §Device tunnel`.

T6 — Certificate deep-check is accurate

Steps: Run `PLAPDX-Check-Certificate.ps1` (optionally `-CaThumbprint <expected>`) and compare to `certlm.msc` (Local Computer → Personal → Certificates). **PASS:** The tool's present / expiry / EKU / chain / private-key / issuer verdicts match what `certlm.msc` shows. **FAIL:** Any verdict disagrees with the store → confirm you ran elevated (private-key visibility needs it) and re-test.

T7 — Intune check finds and correctly scopes objects

Steps: Run `PLAPDX-Check-Intune.ps1` against the tenant. **PASS:** Each expected object (`Pre-Logon VPN – SBL Module – Win32`, `Pre-Logon VPN – SBL – Profile`, `Pre-Logon VPN – Device Tunnel – Profile`, `Pre-Logon VPN – Trusted Root`, `Pre-Logon VPN – Device Cert (SCEP)`) is found, shows its assignment, and the device-tunnel profile is flagged **OK (device group)** — or flagged **MIS-SCOPED** if assigned to a user group. **FAIL:** An object shows MISSING that exists in the portal → confirm it is named to convention verbatim (including the en-dash –).

T8 — Post-fix re-run comes back clean

Steps: After remediating (Phase 3), re-run the focused tool and the collector, then reboot and watch the logon screen / boot. **PASS:** The previously-failing precondition now PASSes, the collector shows no other FAIL, and the device shows the **Network sign-in** tile (SBL) or connects the **device tunnel** before logon. **FAIL:** Still failing → return to the matching branch in `03-implementation.md` / `06-troubleshooting.md`.

Overall exit criteria

The toolkit passes T1–T7 (it reports a known-good device healthy and a known-bad device's true fault), and for a closed incident T8 confirms the failing precondition now PASSes and the device demonstrably brings up pre-logon connectivity. Record results against each gate before closing.

Troubleshooting — Pre-Logon VPN Symptom → Cause → Fix Matrix

This is the heart of the product: a comprehensive **symptom** → **likely cause** → **fix** matrix for pre-logon VPN, covering **both** mechanisms — Cisco **Start Before Logon (SBL/PLAP)** and the Windows Always On VPN **device tunnel**. Work top-down within a section; the first cause listed is the most common. Everything here stays **read-only** on the diagnostic side — the *fix* changes state and is applied in the owning plane (see `03-implementation.md` Phase 3).

> ***Vendor boundary:*** *the **VPN head-end / gateway** internals — the Cisco Secure Client profile editor and ASA/FTD config, and the RRAS / third-party IKEv2 gateway config, including which issuing CA the gateway trusts — are **deferred to the vendor's official documentation and your gateway admin**. This guide resolves the **Windows / Intune / certificate** side and tells you when a failure points at the gateway.*
>
> *Reuse names verbatim:* `Pre-Logon VPN – Device Tunnel – Profile`, `Pre-Logon VPN – SBL Module – Win32`, `Pre-Logon VPN – SBL – Profile`, `Pre-Logon VPN – Trusted Root`, `Pre-Logon VPN – Device Cert (SCEP)`, `PILOT – Pre-Logon VPN Devices`.

§SBL/PLAP — the Network sign-in tile is missing

Symptom: On the Windows logon screen there is **no "Network sign-in" / PLAP option**, so the user can't bring the VPN up before signing in. `PLAPDX-Check-PlapSbl.ps1` shows the first FAIL.

1. **Start Before Logon module not installed.** The tile needs the **SBL module** MSI, and **Core VPN must be installed first**. If `PLAPDX-Check-PlapSbl.ps1` reports the module absent, deploy `Pre-Logon VPN – SBL Module – Win32` (Core VPN as a dependency/first) to the device group and re-sync.

1. `UseStartBeforeLogon` **not set to true.** Even with the module installed, the tile only appears when the Secure Client **VPN profile** carries `<UseStartBeforeLogon>true</UseStartBeforeLogon>`. The check reports the value it found; if it's `false`/absent, set it in `Pre-Logon VPN – SBL – Profile` and re-push.

1. `vpnplap64.dll` **missing or PLAP provider not registered.** The 64-bit PLAP DLL renders the tile and must be registered as a credential provider under `HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Authentication\Credential Providers`. If the DLL is present but the provider isn't registered (or vice-versa), the module install is broken — repair/ reinstall the SBL module so `vpnplap64.dll` registers.

1. **32-bit vs 64-bit mismatch.** On modern 64-bit Windows the PLAP component is `vpnplap64.dll`; a leftover 32-bit `vpnplap.dll` from an old install won't render the tile. Confirm the 64-bit DLL is the one present and registered.

1. **VPN profile not present on the device.** If the Secure Client profile that carries the SBL flag never arrived, there's nothing to read the flag from. Confirm the profile is present (the collector lists it) and that its Intune assignment reached the device.

1. **Wrong mechanism for this device.** If this machine was designed to use the **device tunnel**, the missing SBL tile is expected — run `PLAPDX-Check-DeviceTunnel.ps1` instead (see the next section).

§SBL/PLAP — the tile appears but the connection fails

Symptom: The Network sign-in tile shows, the user picks it, but the VPN won't connect or authentication is rejected.

1. **Certificate rejected (if cert auth).** Run `PLAPDX-Check-Certificate.ps1`. A missing/expired cert, a cert without the **Client Authentication** EKU, or a broken chain is the usual cause — see §Certificate below.

1. **Gateway doesn't trust the issuing CA (vendor side).** If the cert is valid device-side but the head-end still rejects it, the **gateway must trust the issuing CA** — this trust config is **vendor-side, deferred to vendor docs**. Give your gateway admin the issuing-CA name from the certificate check.

1. **Gateway/head-end unreachable pre-logon.** The pre-logon network may not resolve or reach the gateway FQDN. Confirm the machine can reach the head-end from the pre-logon context (a captive portal or missing route breaks this) — a transport problem, not an Intune one.

1. **Secure Client version mismatch.** If the head-end requires a different client version than

installed, align the packaged version and re-pilot. Exact head-end version requirements are vendor-side.

§Device tunnel — never establishes at boot

Symptom: The Always On VPN **device tunnel** doesn't come up before logon; `Get-VpnConnection -AllUserConnection` shows it disconnected or absent. `PLAPDX-Check-DeviceTunnel.ps1` shows the FAIL.

1. **Wrong OS SKU or join type.** The device tunnel requires **Windows Enterprise or Education** and a

domain-joined or hybrid-joined device. On Pro or an Entra-only (not hybrid) join, the device tunnel is unsupported — the check flags this first. Fix the SKU/join, or use SBL instead.

1. **Profile assigned to a user context instead of the device.** The device tunnel connects with **no

user **at boot, so its profile (VPNv2 CSP / ProfileXML) must be** device-scoped **— assigned to a** device** group, not a user group. Run `PLAPDX-Check-Intune.ps1`; if `Pre-Logon VPN – Device Tunnel – Profile` is assigned to a user group, re-assign it to `PILOT – Pre-Logon VPN Devices` (a device group).

1. **More than one device tunnel.** Windows allows **exactly one** device tunnel per device. A second

device tunnel (a leftover or a duplicate profile) makes the tunnel fail. The check counts them; remove the extra.

1. **Not IKEv2.** Device tunnels support **IKEv2 only**. If the profile's connection type is anything

else, reissue `Pre-Logon VPN – Device Tunnel – Profile` as IKEv2.

1. **Missing** `DeviceTunnel=true` **/** `AlwaysOn=true`**.** If the ProfileXML doesn't set both flags, Windows

treats it as an ordinary user tunnel that never auto-connects at boot. The check reads both flags; fix the ProfileXML and re-push.

1. **Machine certificate problem.** The device tunnel authenticates with a **machine certificate** in

`LocalMachine\My` (Client Auth EKU). If it's missing/expired/wrong-EKU, the tunnel can't authenticate — see §Certificate.

§Certificate — the machine / VPN certificate is the fault

Symptom: `PLAPDX-Check-Certificate.ps1` FAILs on one or more facts; the tunnel or SBL connect is rejected at authentication.

1. **No certificate present.** No client-authentication certificate in `LocalMachine\My`. The

trusted root must be deployed first, then the SCEP/PKCS cert. Deploy `Pre-Logon VPN – Trusted Root`, then `Pre-Logon VPN – Device Cert (SCEP)`, and re-sync.

1. **Wrong EKU.** A certificate present but **without the Client Authentication EKU** won't be accepted

for VPN. Fix the certificate template / SCEP profile to include Client Authentication (`1.3.6.1.5.5.7.3.2`).

1. **Expired / not-yet-valid (clock skew).** An in-date cert can appear **not-yet-valid** or **expired**

if the device clock is off. The check flags expiry and not-yet-valid; if the dates look fine but validation still fails on time, check device time sync — this masquerades as a cert fault.

1. **Chain incomplete / untrusted root.** The cert chains to a root the device doesn't trust — the `Pre-Logon VPN – Trusted Root` profile is missing or failed. Deploy/repair it so the chain completes.

1. **No private key.** A cert imported without its key (a PKCS export issue, or SCEP key generation failed) can't be used for authentication. Reissue so the private key is present.

1. **Issued by the wrong CA.** Pass `-CaThumbprint <expected>` to confirm the issuing CA matches the design. A cert from the wrong CA/template will be rejected by a gateway that only trusts the intended issuer.

1. **Valid device-side but rejected at the gateway.** A clean chain locally plus a gateway rejection means the **gateway's trust store** doesn't include your issuing CA (or checks revocation it can't reach). Resolved on the **gateway side** — give the admin the issuer name; **deferred to vendor docs**.

§Cross-cutting — affects either mechanism

1. **Profile assigned to the wrong context.** A device-tunnel profile on a **user** group, or an SBL/cert profile that never applies because the device isn't in the target group. `PLAPDX-Check-Intune.ps1` shows assignment and scope; correct group membership / target.

1. **Domain controller not reachable through the tunnel before logon.** Even with the tunnel/tile up, a first-time user can't log on if **no DC is reachable** over the pre-logon path (the tunnel's routes don't include the DC subnets, or DNS for the domain doesn't resolve pre-logon). This is a routing/DNS design issue on the tunnel/gateway side.

1. **Device not synced / not enrolled.** Nothing (module, profile, cert) applies to a device that isn't enrolled and syncing. Confirm enrollment and force a sync, then re-collect.

1. **Naming drift.** The Intune check matches objects **by exact display name**. If an object was renamed off the convention (including the en-dash – and spacing), it shows MISSING though it exists. Rename it back to convention.

§Script / tooling issues

1. **Access denied reading** `LocalMachine\My` **or the Credential Providers key.** Run the device-side scripts **elevated** (R2). Read-only, but elevation is needed for visibility.

1. `Connect-MgGraph` **fails or lacks scopes.** Install the SDK (`Install-Module Microsoft.Graph -Scope CurrentUser`) and consent to the read scopes PLAPDX-Check-Intune.ps1` lists in its header.

1. `Get-VpnConnection -AllUserConnection` **returns nothing.** Either there is genuinely no device tunnel (a valid finding — see §Device tunnel) or you didn't run elevated; confirm elevation first.

When to stop diagnosing and escalate to the gateway vendor

Diagnosis is read-only by design. When the **Windows / Intune / certificate** side is confirmed clean (the focused checks PASS and the collector shows no FAIL) but the tunnel still won't authenticate, treat it as a **gateway / head-end issue** and continue with the **vendor's official documentation** and your gateway admin. Hand over: the certificate check output (issuer, EKU, validity, chain), a statement that the device side is clean, and the exact rejection time so the admin can find the attempt in the gateway log. A complete package turns a multi-round case into a single decisive one.

Best Practices — Repeatable Pre-Logon VPN Triage

The Diagnostic Workflow isolates a missing precondition in one incident. These practices make that repeatable across incidents and across the team, so every triage is faster and cleaner than the last. Everything here stays **read-only** and vendor-boundary-aware.

> *Reuse names verbatim:* `Pre-Logon VPN – Device Tunnel – Profile`, `Pre-Logon VPN – SBL Module – Win32`, `Pre-Logon VPN – SBL – Profile`, `Pre-Logon VPN – Trusted Root`, `Pre-Logon VPN – Device Cert (SCEP)`, `PILOT – Pre-Logon VPN Devices`.

1. Name the intended mechanism before you touch anything

• Every pre-logon device should be **designed for one mechanism** — Cisco SBL/PLAP **or** the AOVPN device tunnel — not "whichever happens to be on it." The single biggest time-saver is confirming the intended mechanism (R5) before diagnosing, so you don't "fix" a device tunnel on a machine that was only ever meant to use SBL.

• Standardize the choice per device population (e.g. device tunnel for domain/hybrid Enterprise fleets; SBL where the head-end and client demand it) and record it with the deployment.

2. Collect first, reason second

• Run `PLAPDX-Collect-Diagnostics.ps1` **once, early** — snapshot the whole device state before you change anything, so you have a before/after and don't lose the failing state.

• Prefer the scripts over click-by-click checks for anything you'll do more than once: they're consistent, and the collector's `-OutFile` report attaches straight to the ticket.

• Snapshot **both mechanisms** even when you think you know which one is at fault — the collector frequently reveals a second, quieter gap (e.g. a machine cert that's also expired) that would have caused a repeat incident.

3. Check preconditions in a fixed order

• **SBL:** module installed (Core VPN first) → `vpnplap64.dll` present → PLAP provider registered →

profile present → `UseStartBeforeLogon=true`. The first FAIL is almost always the whole reason.

- **Device tunnel:** SKU/join eligible → exactly one device tunnel → IKEv2 → `DeviceTunnel/AlwaysOn`

set → machine cert valid. Don't chase the cert while the SKU/join is unverified.

- Treat a rejected connection as a pointer to the **certificate** first, and the **gateway trust

store** only after the device-side cert checks clean.

4. Keep a known-good baseline current

- Maintain at least one working reference device (R6), ideally from `PILOT – Pre-Logon VPN Devices`,

and record its **issuer, template, profile name, and gateway**. Diagnosis is comparison; a baseline turns open questions into diffs.

- Refresh the baseline whenever the CA, gateway, trusted root, or VPN profile changes. A stale baseline

lies.

5. Get the assignment scope right — device vs user

- The **device-tunnel profile must target a device group**, never a user group — the tunnel connects

with no user at boot. Audit this with `PLAPDX-Check-Intune.ps1` on a schedule; a mis-scoped profile is the single most common silent device-tunnel failure.

- Keep SBL module + SBL profile + cert profiles assigned to the **same** device population, so a device

is either fully pre-logon-ready or clearly not — never half-configured.

6. Read-only discipline

- Diagnostics **observe; they never remediate.** No script here changes device, tenant, or gateway

state. This keeps triage safe to run in production, on a user's device, mid-incident, without a change window.

- When you *do* remediate, do it in the owning plane (the Intune app/profile/cert, or a gateway-side

change by the gateway admin) — deliberately, under change control for anything shared or outward-facing (the issuing CA, the gateway, a broadly assigned profile).

7. Respect the vendor boundary

- For the **VPN head-end / gateway** (Cisco ASA/FTD, RRAS, or a third-party IKEv2 gateway) and the

Cisco Secure Client **profile editor**, gather the evidence the checks name (the issuing-CA name, the rejection reason, whether the attempt arrived) and read it — but follow the **vendor's official documentation** for exact console navigation and trust-store configuration. Don't invent menu paths.

- Author every check on the **Windows / Intune / Entra / certificate** side at exact detail — that's

where your leverage and precision are.

8. Least-privilege, auditable access

• Use **read-only** roles (Global Reader / a custom read role) for portal work, and **read scopes only **for Graph. The toolkit never needs write access. Run the device-side scripts** elevated** for visibility (R2), not to change anything.

9. Close the loop

• After every incident, confirm with the matching **gate in** `05-testing.md` and reboot to see the tile appear / the tunnel connect — a fix isn't done until pre-logon connectivity demonstrably works.

• Record the **root-cause precondition** on the ticket (module / flag / provider / SKU-join / single-tunnel / IKEv2 / cert / scope). Patterns emerge: repeated cert failures mean the trusted-root or SCEP profile needs attention; repeated device-tunnel mis-scopes mean the assignment process needs a guardrail. Fix the pattern, not just the incident.

Deliverables Summary

Everything included in the **Pre-Logon VPN (PLAP / Start Before Logon) Troubleshooting & Diagnostics Toolkit**, and how each piece is used. This is a diagnostics product: a reusable, self-guided workflow plus read-only scripts you keep and run on every pre-logon VPN incident — covering **both** Cisco Start Before Logon (SBL/PLAP) and the Windows Always On VPN device tunnel.

Documentation

Document	What it gives you	When you use it
Solution Overview (`01-solution-overview.md`)	The two pre-logon mechanisms, their preconditions, the five tools, and the read-only/safe constraint	Read once to understand the stack; refer back when a check mentions a precondition
Requirements (`02-requirements.md`)	The device access, admin rights, Graph/Intune read access, and vantage points to diagnose, each with Description / Why / Verify / Implement-if-missing	Before an incident — assemble access so you're not gathering it mid-outage
Diagnostic Workflow (`03-implementation.md`)	The ordered **Collect** → **Analyze** → **Remediate** → **Verify** workflow and the symptom → tool map	The core deliverable — start here during an incident
Testing (`05-testing.md`)	Pass/fail gates that the toolkit reports a known-good device healthy and a known-bad device's true fault, plus post-fix confirmation	To trust the tool's verdicts, and after a fix

Document	What it gives you	When you use it
Troubleshooting (`06-troubleshooting.md`)	The comprehensive symptom → cause → fix matrix for pre-logon VPN across both mechanisms	The heart of the product — during an incident, and for edge cases
Best Practices (`07-best-practices.md`)	Repeatable triage order, evidence collection, correct assignment scope, least-privilege access, closing the loop	To standardize triage across the team and across incidents
Executive Summary (`executive-summary.md`)	The problem, what's inside, and the value, for a decision-maker	To brief a stakeholder or justify the toolkit
Global Requirements — Network (`global-requirements-network.md`)	Category-level prerequisites shared by all Network solutions	Confirm alongside the solution-specific requirements

PowerShell scripts (all read-only)

Script	What it does	Area it covers
PLAPDX-Collect-Diagnostics.ps1	The entry-point master collector — snapshots every precondition for **both** mechanisms (OS SKU/build, join type, Secure Client version, `vpnplap64.dll` + PLAP registration, `UseStartBeforeLogon`, the device tunnel + flags, the machine cert) into one console + `-OutFile` report	Both mechanisms
PLAPDX-Check-PlapSbl.ps1	Focused SBL/PLAP checks with PASS/FAIL per precondition (module installed, DLL present, provider registered, `UseStartBeforeLogon=true`, profile present) and the exact remedy for each FAIL	SBL / PLAP
PLAPDX-Check-DeviceTunnel.ps1	Focused device-tunnel checks (SKU/join eligible, exactly one device tunnel, IKEv2, machine cert valid, AlwaysOn) with PASS/FAIL + remedy	AOVPN device tunnel
PLAPDX-Check-Certificate.ps1	Machine-certificate deep-check (present, not expired, Client Auth EKU, chains to a trusted root, private key), optional `-CaThumbprint` to confirm the expected issuing CA	Certificate (both mechanisms)

Script	What it does	Area it covers
PLAPDX-Check-Intune.ps1	Read-only Graph audit — confirms the expected Intune objects exist, are assigned, and flags a device-tunnel profile mis-scoped to a **user** group	Intune objects + scoping

Every script begins with a standard PURPOSE / HOW TO RUN / EXPECTED RESULTS header. The four device-side scripts need **no** Graph access (run them **elevated** for full visibility); the Intune check requires only **read** Graph scopes. None change the device, tenant, or gateway.

What is intentionally deferred to the gateway vendor

The **VPN head-end / gateway** internals — the Cisco Secure Client profile editor and ASA/FTD config, the RRAS / third-party IKEv2 gateway config, and which issuing CA the gateway trusts — are **deferred to the vendor's official documentation**. The toolkit names the exact evidence to read on the gateway side (did the attempt arrive, the rejection reason, the issuing-CA name to trust) but does not fabricate vendor menu paths. Remediation of a device-side finding is pointed at the matching **deployment** product; the toolkit itself stays read-only.

How the pieces fit together

1. **Prepare** with `02-requirements.md`.
2. **Collect** with `PLAPDX-Collect-Diagnostics.ps1`, then **Analyze** with the focused `Check-*` tool

the symptom points to (`03-implementation.md`).

1. **Remediate** in the owning plane, then **Verify** by re-running the tools and rebooting.
2. **Confirm** with `05-testing.md`; deep-dive edge cases with `06-troubleshooting.md`.
3. **Standardize** with `07-best-practices.md`.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

PLAPDX-Check-Certificate.ps1

Purpose: For each candidate machine certificate in LocalMachine\My, checks the facts pre-logon VPN authentication depends on: present, not expired (and not yet-valid), Client Authentication EKU, chains to a trusted root, and a private key present. Optionally confirm the expected issuing CA with -CaThumbprint. This is the certificate that backs both the AOVPN device tunnel and (where cert auth is used) the SBL/PLAP connect. Read-only: it inspects, never modifies, the store. HOW TO RUN Run ELEVATED (private-key visibility for LocalMachine\My needs it): powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-Certificate.ps1 powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-Certificate.ps1 -CaThumbprint <expected-issuing-CA-thumbprint> No Microsoft Graph scopes needed — this is a local device check. EXPECTED RESULTS A per-certificate PASS/FAIL table (private key, expiry, Client Auth EKU, chain, and — with -CaThumbprint — issued-by-expected-CA). At least one PASS => the cert side is healthy; if the gateway still rejects it, the head-end likely does not trust the issuing CA (vendor-deferred). No client-auth cert / none pass => fix the trusted root + SCEP cert (see 06-troubleshooting.md). .NOTES White-label — the repeatable way. Read-only local inspection. VPN head-end / gateway internals deferred to vendor docs. >

How to run: Run ELEVATED (private-key visibility for LocalMachine\My needs it): powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-Certificate.ps1 powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-Certificate.ps1 -CaThumbprint <expected-issuing-CA-thumbprint> No Microsoft Graph scopes needed — this is a local device check. EXPECTED RESULTS A per-certificate PASS/FAIL table (private key, expiry, Client Auth EKU, chain, and — with -CaThumbprint — issued-by-expected-CA). At least one PASS => the cert side is healthy; if the gateway still rejects it, the head-end likely does not trust the issuing CA (vendor-deferred). No client-auth cert / none pass => fix the trusted root + SCEP cert (see 06-troubleshooting.md). .NOTES White-label — the repeatable way. Read-only local inspection. VPN head-end / gateway internals deferred to vendor docs. >

PLAPDX-Check-DeviceTunnel.ps1

Purpose: Checks, with PASS/FAIL per precondition, everything a device tunnel needs to establish before logon: an eligible OS SKU (Windows Enterprise/Education) and join type (domain-joined or hybrid-joined), EXACTLY ONE device tunnel (only one is allowed per device), the connection is IKEv2, DeviceTunnel=true and AlwaysOn=true are set, and a valid machine certificate backs it. Prints the exact remedy for each FAIL. Read-only: it never starts, stops, or reconfigures a tunnel — it reads Get-VpnConnection -AllUserConnection, the OS/join state, and the cert store. HOW TO RUN Run ELEVATED (needs to read all-user VPN connections and LocalMachine\My): powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-DeviceTunnel.ps1 No Microsoft Graph scopes needed — this inspects the local device only. EXPECTED RESULTS A PASS/FAIL line per precondition with a remedy for each FAIL (fix SKU/join, remove the extra device tunnel, reissue as IKEv2, set the flags, fix the machine cert). All PASS => the device tunnel should connect at boot (see 06-troubleshooting.md). .NOTES White-label — the repeatable way. Read-only local inspection. VPN head-end / gateway internals deferred to vendor docs. >

How to run: Run ELEVATED (needs to read all-user VPN connections and LocalMachine\My): powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-DeviceTunnel.ps1 No Microsoft Graph scopes needed —

this inspects the local device only. EXPECTED RESULTS A PASS/FAIL line per precondition with a remedy for each FAIL (fix SKU/join, remove the extra device tunnel, reissue as IKEv2, set the flags, fix the machine cert). All PASS => the device tunnel should connect at boot (see 06-troubleshooting.md). .NOTES White-label — the repeatable way. Read-only local inspection. VPN head-end / gateway internals deferred to vendor docs. >

PLAPDX-Check-Intune.ps1

Purpose: Read-only Microsoft Graph audit for a pre-logon VPN deployment. Confirms the expected Intune objects are present and assigned — the SBL module Win32 app, the SBL-enabling VPN profile, the device-tunnel ProfileXML custom profile, the trusted-root profile, and the SCEP/PKCS cert profile — and flags the classic mis-scope: a DEVICE-TUNNEL profile assigned to a USER group instead of a DEVICE group (the tunnel connects with no user at boot, so it must target a device group). Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-Intune.ps1 Required Microsoft Graph scopes (delegated, read-only): DeviceManagementApps.Read.All DeviceManagementConfiguration.Read.All Group.Read.All Directory.Read.All EXPECTED RESULTS A table: each expected object Found/Missing with its assignment state, and for the device-tunnel profile an OK (device group) / MIS-SCOPED (user group) verdict. All Found + assigned + device tunnel on a device group = healthy. Missing/UNASSIGNED/MIS-SCOPED = fix (see 03-implementation.md). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. VPN head-end / gateway internals deferred to vendor docs. >

How to run: powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-Intune.ps1 Required Microsoft Graph scopes (delegated, read-only): DeviceManagementApps.Read.All DeviceManagementConfiguration.Read.All Group.Read.All Directory.Read.All EXPECTED RESULTS A table: each expected object Found/Missing with its assignment state, and for the device-tunnel profile an OK (device group) / MIS-SCOPED (user group) verdict. All Found + assigned + device tunnel on a device group = healthy. Missing/UNASSIGNED/MIS-SCOPED = fix (see 03-implementation.md). .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. VPN head-end / gateway internals deferred to vendor docs. >

PLAPDX-Check-PlapSbl.ps1

Purpose: Checks, with PASS/FAIL per precondition, everything the Windows "Network sign-in" (PLAP) tile needs to appear: the Start Before Logon module installed (Core VPN first), the 64-bit PLAP component vpnplap64.dll present, the PLAP credential provider registered under HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Authentication\Credential Providers, the Cisco Secure Client VPN profile present, and <UseStartBeforeLogon>true</UseStartBeforeLogon> set in that profile. Prints the exact remedy for each FAIL. Read-only: it never changes anything. HOW TO RUN Run ELEVATED (needs to read the Credential Providers registry key): powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-PlapSbl.ps1 No Microsoft Graph scopes needed — this inspects the local device only. EXPECTED RESULTS A PASS/FAIL line per precondition. The first FAIL is almost always why the tile is missing; each FAIL prints its remedy (deploy the SBL module, set UseStartBeforeLogon, repair the module so vpnplap64.dll registers). All PASS => the tile should appear (see 06-troubleshooting.md). .NOTES White-label — the repeatable way. Read-only local inspection. VPN head-end / gateway internals deferred to vendor docs. >

How to run: Run ELEVATED (needs to read the Credential Providers registry key): powershell -ExecutionPolicy Bypass -File .\PLAPDX-Check-PlapSbl.ps1 No Microsoft Graph scopes needed — this

inspects the local device only. EXPECTED RESULTS A PASS/FAIL line per precondition. The first FAIL is almost always why the tile is missing; each FAIL prints its remedy (deploy the SBL module, set UseStartBeforeLogon, repair the module so vpnplap64.dll registers). All PASS => the tile should appear (see 06-troubleshooting.md). .NOTES White-label — the repeatable way. Read-only local inspection. VPN head-end / gateway internals deferred to vendor docs. >

PLAPDX-Collect-Diagnostics.ps1

Purpose: The entry point. Read-only, on-device snapshot of EVERY precondition for both pre-logon VPN mechanisms into one report, so triage is spent reasoning instead of gathering. Collects: OS SKU + build; join type (dsregcmd); Cisco Secure Client version; vpnplap64.dll presence + PLAP credential-provider registration; the UseStartBeforeLogon value in the Secure Client VPN profile; the Always On VPN device tunnel (Get-VpnConnection -AllUserConnection) with its DeviceTunnel/AlwaysOn/TunnelType; and the machine certificate (LocalMachine\My: EKU + expiry + private key). Ends with a flagged mechanism + symptom. Makes NO changes. HOW TO RUN Run ELEVATED (needed to read LocalMachine\My, all-user VPN, and the Credential Providers key): powershell -ExecutionPolicy Bypass -File .\PLAPDX-Collect-Diagnostics.ps1 powershell -ExecutionPolicy Bypass -File .\PLAPDX-Collect-Diagnostics.ps1 -OutFile .\plapdx-report.txt No Microsoft Graph scopes needed — this inspects the local device only. EXPECTED RESULTS A console summary and (with -OutFile) a text report of every precondition, plus a flagged mechanism/symptom pointing at the focused Check-* tool to run next (see 03-implementation.md). .NOTES White-label — the repeatable way. Read-only local inspection. VPN head-end / gateway internals deferred to vendor docs. >

How to run: Run ELEVATED (needed to read LocalMachine\My, all-user VPN, and the Credential Providers key): powershell -ExecutionPolicy Bypass -File .\PLAPDX-Collect-Diagnostics.ps1 powershell -ExecutionPolicy Bypass -File .\PLAPDX-Collect-Diagnostics.ps1 -OutFile .\plapdx-report.txt No Microsoft Graph scopes needed — this inspects the local device only. EXPECTED RESULTS A console summary and (with -OutFile) a text report of every precondition, plus a flagged mechanism/symptom pointing at the focused Check-* tool to run next (see 03-implementation.md). .NOTES White-label — the repeatable way. Read-only local inspection. VPN head-end / gateway internals deferred to vendor docs. >